

zomato



Strategic Insights: Zomato Business Model & Profitability

EXPLORATORY DATA ANALYSIS (EDA)

INTRODUCTION TO ZOMATO PROJECT

Zomato is a globally recognized platform that connects users with a wide array of dining options, ranging from local restaurants to high-end eateries. Founded in 2008, Zomato has evolved from being a restaurant discovery platform to a comprehensive service offering that includes food delivery, restaurant reservations, reviews, and more.

This project focuses on exploring and enhancing the Zomato platform, analyzing its data, and identifying innovative solutions to improve user experience and business operations. It involves studying user interaction with the platform, tracking restaurant ratings, exploring delivery logistics, and

integrating features that can make Zomato a more efficient and intuitive service for customers and businesses alike.

The aim of the Zomato project is to examine the key elements that contribute to Zomato's success, analyse user data for insights, and develop solutions that can optimize restaurant discovery, improve food delivery systems, and further enhance customer satisfaction. Through this project, the goal is to create actionable insights, improve functionality, and propose creative ideas that help in scaling Zomato's services in an increasingly competitive food-tech industry.

OBJECTIVE OF ZOMATO ANALYSIS

The objective of Zomato analysis is to deeply understand the dynamics of the platform, identify patterns, and derive actionable insights that can enhance user experience, business operations, and overall service efficiency. Specifically, the analysis aims to achieve the following:

1.Customer Behaviour Analysis:

- Understand user preferences, browsing habits, and food ordering trends.
- Analyse customer reviews, ratings, and feedback to gauge satisfaction levels and identify areas for improvement.

2.Restaurant Performance Assessment:

- Evaluate the performance of restaurants on Zomato, including customer ratings, delivery time, menu quality, and service efficiency.
- Identify top-performing restaurants and analyze the factors that contribute to their success.

3.Food Delivery Efficiency:

- Analyze food delivery times and assess factors affecting delivery efficiency, such as distance, time of day, and restaurant type.

- Identify potential bottlenecks in the delivery process and suggest improvements for faster and more reliable deliveries.

4. Market Trend Insights:

- Identify and track emerging food trends based on search data, orders, and user preferences.
- Explore regional differences in food preferences and ordering habits, helping businesses tailor their offerings accordingly.

5. Revenue and Profitability Analysis:

- Analyze the financial performance of restaurants and delivery services on the platform.
- Understand the impact of discounts, promotions, and loyalty programs on customer acquisition, retention, and revenue.

6. Predictive Analytics for Business Growth:

- Utilize predictive modelling to forecast demand, customer behaviour, and market trends.
- Provide actionable recommendations to improve customer acquisition, retention, and operational efficiency.

7. Competitor Benchmarking:

- Analyze how Zomato compares with its competitors in the food delivery and restaurant discovery space.
- Identify gaps in the market and suggest ways Zomato can enhance its competitive edge.

8. IMPROVING USER INTERFACE AND EXPERIENCE:

- Analyze how users interact with the platform and identify areas where the user interface can be optimized for better experience and convenience.

By achieving these objectives, the Zomato analysis will help in refining the platform's offerings, improving customer satisfaction, and driving sustainable business growth.

IMPORTING PYTHON LIBRARIES

NUMPY [IMPORT NUMPY AS NP] :- NumPy is a Python library that provides support for efficient multi-dimensional arrays and matrices. Its core feature is the ndarray object, which allows fast operations on large datasets. NumPy offers a wide range of mathematical functions like linear algebra, statistics, and element-wise operations. It enables vectorized computations, significantly improving performance over traditional Python lists. NumPy is foundational for scientific computing and integrates seamlessly with other libraries like Pandas and SciPy.

PANDAS [IMPORT PANDAS AS PD]:- Pandas is a powerful Python library used for data manipulation and analysis, providing easy-to-use data structures like Data Frames and Series. It simplifies handling and processing of structured data, such as CSV files, SQL databases, and Excel spreadsheets. Pandas allows fast and flexible operations like filtering, grouping, and aggregating data. It also integrates well with other data analysis libraries, such as NumPy and Matplotlib. With its rich functionality, Pandas is widely used in data science, machine learning, and statistical analysis.

MATPLOTLIB [IMPORT MATPLOTLIB AS NP] :- Matplotlib is a popular Python library used for creating static, interactive, and animated visualizations. It provides a wide range of plotting capabilities, such as line plots, bar charts,

histograms, scatter plots, and more. Matplotlib is highly customizable, allowing users to fine-tune every aspect of a plot, including colors, labels, and styles. It integrates seamlessly with other libraries like Pandas and NumPy for data visualization. As a fundamental tool in data science, Matplotlib helps in visually interpreting and analyzing data insights.

SEABORN [IMPORT SEABORN AS SNP] :- Seaborn is a powerful Python visualization library built on top of Matplotlib, designed for creating attractive and informative statistical graphics. It simplifies the creation of complex plots like heatmaps, violin plots, and pair plots with just a few lines of code. Seaborn integrates seamlessly with Pandas DataFrames, making it easy to visualize data directly from structured datasets. It provides built-in themes and color palettes to improve the aesthetics of plots. Seaborn is widely used in data science for exploring and visualizing data distributions and relationships.

PLOTLY [IMPORT PLOTLY.EXPRESS AS PX] :- Plotly is an interactive graphing library for Python that enables the creation of dynamic, web-based visualizations. It supports a wide variety of charts, including 3D plots, scatter plots, line charts, and geographic maps. Plotly allows for seamless interactivity, such as zooming, panning, and tooltips, enhancing the user experience. It integrates well with other libraries like Pandas and Matplotlib and can be easily embedded into web applications. Widely used in data science and business analytics, Plotly provides both offline and online capabilities for visualization sharing and collaboration.

DATA OVERVIEW

LOADING DATASET AND LIBRARIES

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import plotly.express as px

df=pd.read_csv("/content/drive/MyDrive/dataset_3/Indian-Resturants.csv")
df
```

I have load dataset named “Indian_restaurant.csv” into data overview a variable named “df”.

To import the dataset. I have used Pandas Libraries in which I have used “pd.read_csv”

Command to import the respective dataset.

DF.DESCRIBE(): The describe() function in Pandas provides a quick summary of the statistics of numerical columns in a DataFrame. It returns key metrics such as count, mean, standard deviation, minimum, maximum, and quartiles (25%, 50%, and 75%). This function is useful for understanding the distribution and spread of data at a glance. By default, it works on numeric columns, but can be customized to include other data types. describe() is a handy tool for initial exploratory data analysis (EDA) to assess the dataset's characteristics.

```
df.describe()
```

	res_id	city_id	latitude	longitude	country_id	average_cost_for_two	price_range	aggregate_rating	votes	photo_count	delivery	takeaway
count	6.041700e+04	60417.000000	60417.000000	60417.000000	60417.0	60417.000000	60417.000000	60417.000000	60417.000000	60417.000000	60417.000000	60417.0
mean	1.309335e+07	3418.302183	21.349431	76.588040	1.0	538.304517	1.730821	3.032868	261.574888	194.247414	-0.371799	-1.0
std	8.132809e+06	5179.351720	41.187998	10.600514	0.0	593.852227	0.880462	1.440751	728.284194	705.682451	0.925249	0.0
min	5.000000e+01	1.000000	0.000000	0.000000	1.0	0.000000	1.000000	0.000000	-18.000000	0.000000	-1.000000	-1.0
25%	3.000488e+06	7.000000	16.324755	74.654028	1.0	200.000000	1.000000	2.900000	7.000000	1.000000	-1.000000	-1.0
50%	1.869150e+07	26.000000	22.320884	77.135310	1.0	400.000000	1.000000	3.500000	42.000000	11.000000	-1.000000	-1.0
75%	1.886666e+07	11295.000000	26.744389	79.928190	1.0	600.000000	2.000000	4.000000	207.000000	82.000000	1.000000	-1.0
max	1.915979e+07	11354.000000	10000.000000	91.832769	1.0	30000.000000	4.000000	4.900000	42539.000000	17702.000000	1.000000	-1.0

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 60417 entries, 0 to 211942
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   res_id                60417 non-null  int64  
1   name                  60417 non-null  object  
2   establishment          60417 non-null  object  
3   url                   60417 non-null  object  
4   address               60417 non-null  object  
5   city                  60417 non-null  object  
6   city_id               60417 non-null  int64  
7   locality              60417 non-null  object  
8   latitude              60417 non-null  float64 
9   longitude             60417 non-null  float64 
10  country_id            60417 non-null  int64  
11  locality_verbose      60417 non-null  object  
12  cuisines              60417 non-null  object  
13  timings               60417 non-null  object  
14  average_cost_for_two  60417 non-null  int64  
15  price_range           60417 non-null  int64  
16  currency              60417 non-null  object  
17  highlights            60417 non-null  object  
18  aggregate_rating      60417 non-null  float64 
19  rating_text           60417 non-null  object  
20  votes                 60417 non-null  int64  
21  photo_count           60417 non-null  int64  
22  opentable_support     60417 non-null  object  
23  delivery              60417 non-null  int64  
24  takeaway              60417 non-null  int64  
dtypes: float64(3), int64(9), object(13)
memory usage: 12.0+ MB
```

The `df.info()` function in Pandas provides a concise summary of a `DataFrame`, including the number of entries, column names, data types, and memory usage. It helps identify missing data by showing non-null counts for each column. This function is essential for quickly understanding the structure of a dataset, especially its size and data types. It allows users to assess whether data cleaning or preprocessing is needed. `df.info()` is a useful tool during the initial stages of data exploration and analysis.

CHECKING DIMENSIONS

```
df.shape
```

```
(60417, 25)
```

Df.shape:- The `df.shape` attribute in Pandas returns a tuple (rows=60417, columns=25), indicating the dimensions of the `DataFrame`. It provides a quick way to assess the size of the dataset.

CHECKING AND DROP DUPLICATE

```
df.duplicated().sum()
```

```
151527
```

```
df.drop_duplicates(subset=None,keep="first",inplace=True)
```

The `duplicated()` function in Pandas returns a Boolean Series indicating whether each row in a DataFrame is a duplicate of a previous row. It helps identify duplicates, which can be used for data cleaning or analysis. The `drop_duplicates()` function, on the other hand, removes duplicate rows from a DataFrame, keeping the first (or last) occurrence by default. Both functions allow customization through parameters like `subset` (to specify columns) and `keep` (to choose which duplicates to keep). Together, they are essential tools for managing redundant data in datasets.

CALCULATE PERCENTAGE OF NAN VALUE AND MISSING VALUE.

The code `df.isnull().sum()/df.shape[0]*100` is used to calculate the percentage of missing values (NaN) in each column of a pandas DataFrame (`df`). First, `df.isnull()` identifies the missing values, then `sum()` counts them for each column. By dividing this count by the total number of rows (`df.shape[0]`), we get the fraction of missing values in each column. Finally, multiplying by 100 converts the fraction into a percentage. This helps in quickly assessing the extent of missing data in a DataFrame.

```
df.isnull().sum()/df.shape[0]*100
```

```
address
```

```
0.029793
```


zipcode	79.231011
country_id	0.000000
locality_verbose	0.000000
cuisines	0.777927
timings	1.771025

"HANDLING MISSING DATA IN PANDAS: REPLACING NAN VALUES WITH DEFAULT STRINGS"

The code replaces missing (NaN) values in specific columns of a pandas DataFrame (df) with placeholder strings. The fillna() method is used to fill missing values, and inplace=True ensures the changes are made directly to the original DataFrame. Specifically, the missing values in the address, cuisines, timings, and opentable_support columns are replaced with "Unknown" or "unknown" as appropriate. This approach helps avoid issues caused by missing data in these columns by providing default values.

```
df["address"].fillna("Unknown",inplace=True)
df["cuisines"].fillna("Unknown",inplace=True)
df["timings"].fillna("unknown",inplace=True)
df["opentable_support"].fillna("unknown",inplace=True)
```

CALCULATE AND VISUALIZE THE AVERAGE RATING OF RESTAURANTS. ANALYZE THE DISTRIBUTION OF RESTAURANT RATINGS TO UNDERSTAND THE OVERALL RATING LANDSCAPE.

```
a=df.groupby("city")['aggregate_rating'].mean().reset_index()
display(a)
```

	city	aggregate_rating
0	Agra	2.745404
1	Ahmedabad	3.266591
2	Ajmer	2.790426
3	Alappuzha	0.525843
4	Allahabad	2.771252
...	...	...
94	Varanasi	3.037793
95	Vellore	2.301466
96	Vijayawada	3.066226
97	Vizag	3.189182
98	Zirakpur	2.827273
99 rows × 2 columns		

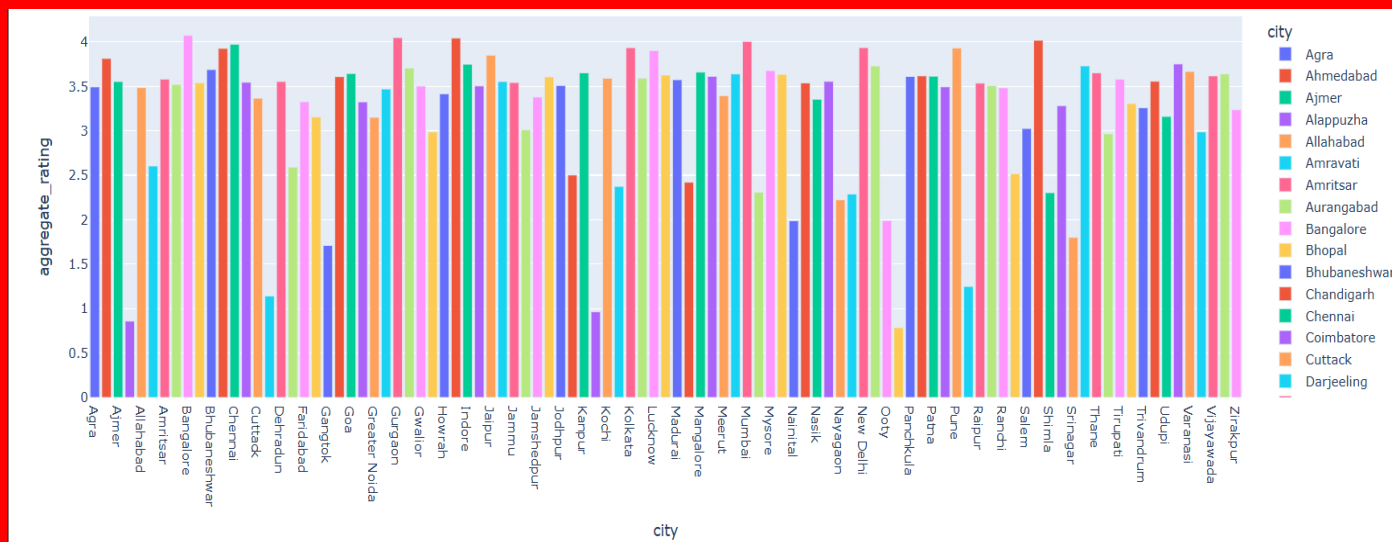
THIS CODE SNIPPET USES PLOTLY EXPRESS TO CREATE A BAR CHART. THE `PX.BAR()` FUNCTION IS USED TO GENERATE THE BAR CHART WITH THE FOLLOWING PARAMETERS:

1. `a` is the DataFrame that holds the data.
2. `x="city"` specifies that the x-axis will represent the "city" column.
3. `y="aggregate_rating"` sets the y-axis to represent the "aggregate_rating" column.
4. `color="city"` assigns different colors to each bar based on the city.
5. `b.show()` displays the chart.

```

b=px.bar(a,x="city",y="aggregate_rating",color="city")
b.show()
plt.figure(figsize=(10, 6))
plt.figure(figsize=(10, 6))
sns.histplot(a['aggregate_rating'], bins=20, kde=True)
plt.title('Distribution of Restaurant Ratings')
plt.xlabel('Aggregate Rating')
plt.ylabel('Frequency')
plt.show()

```

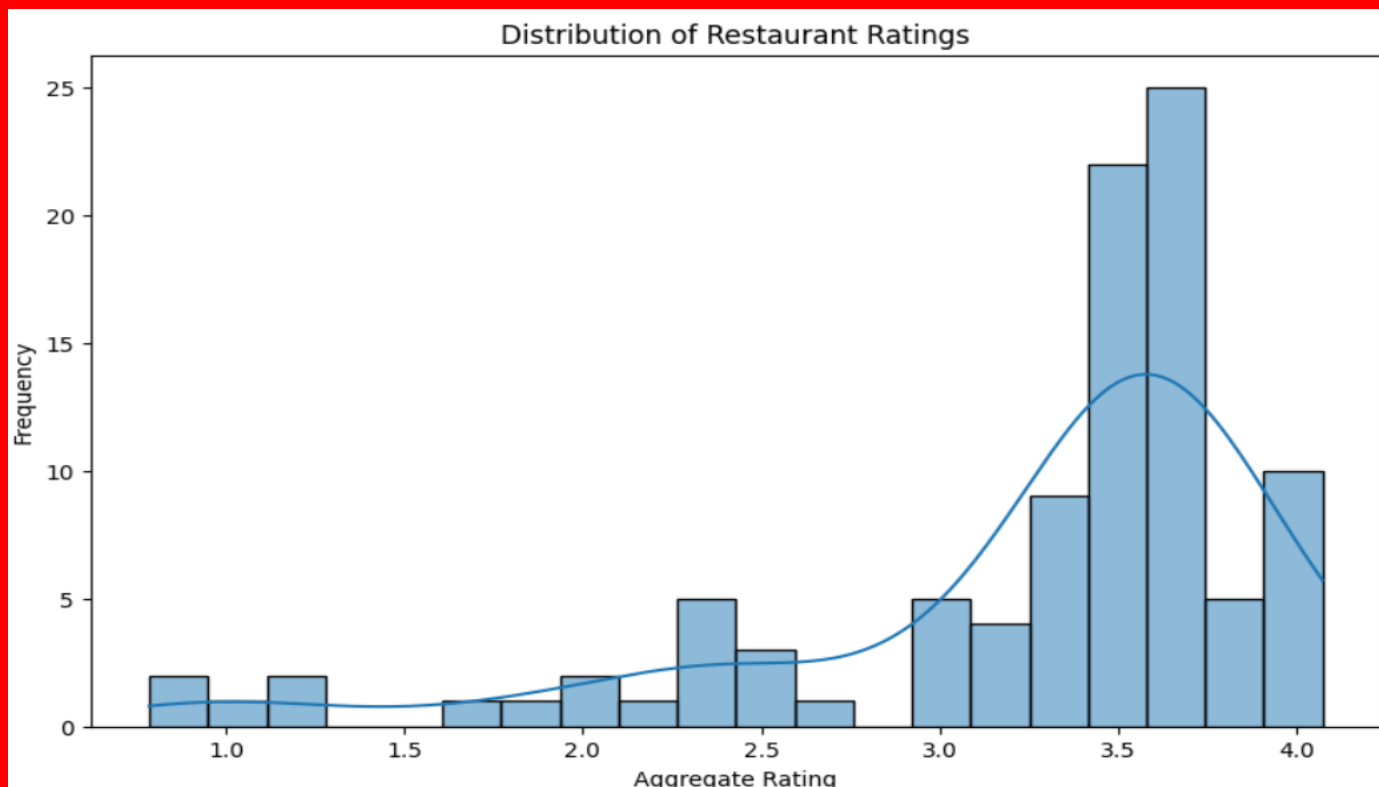


THIS VISUALIZATION HELPS ANALYZE THE FREQUENCY AND DISTRIBUTION OF RESTAURANT RATINGS.

THIS CODE SNIPPET USES SEABORN AND MATPLOTLIB TO VISUALIZE THE DISTRIBUTION OF RESTAURANT RATINGS.

- I. `sns.histplot()` creates a histogram of the "aggregate_rating" column from DataFrame a.

2. `bins=20` sets the number of bins to 20 for the histogram.
3. `kde=True` adds a Kernel Density Estimate curve to the plot for a smoother representation of the data.
4. The title and labels are added using `plt.title()`, `plt.xlabel()`, and `plt.ylabel()`.
5. `plt.show()` displays the plot.



LOCATION ANALYSIS:

IDENTIFY THE CITY WITH THE HIGHEST CONCENTRATION OF RESTAURANTS.

```
restaurant_counts = df.groupby('city')['name'].count().reset_index()

restaurant_counts.rename(columns={'name': 'restaurant_count'}, inplace=True)

city_with_highest_concentration = restaurant_counts.loc[restaurant_counts['restaurant_count'].idxmax()]

print(city_with_highest_concentration)
```

The city with the highest concentration of restaurants is: Chennai
Number of restaurants: 11630

VISUALIZE THE DISTRIBUTION OF RESTAURANT RATINGS ACROSS DIFFERENT CITIES

```
city_ratings = df.groupby('city')['aggregate_rating'].agg(['count', 'mean', 'median', 'std'])

city_ratings = city_ratings.reset_index()

plt.figure(figsize=(12, 6))
sns.boxplot(x='city', y='aggregate_rating', data=df)
plt.title('Distribution of Restaurant Ratings Across Cities')
plt.xlabel('City')
plt.ylabel('Aggregate Rating')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

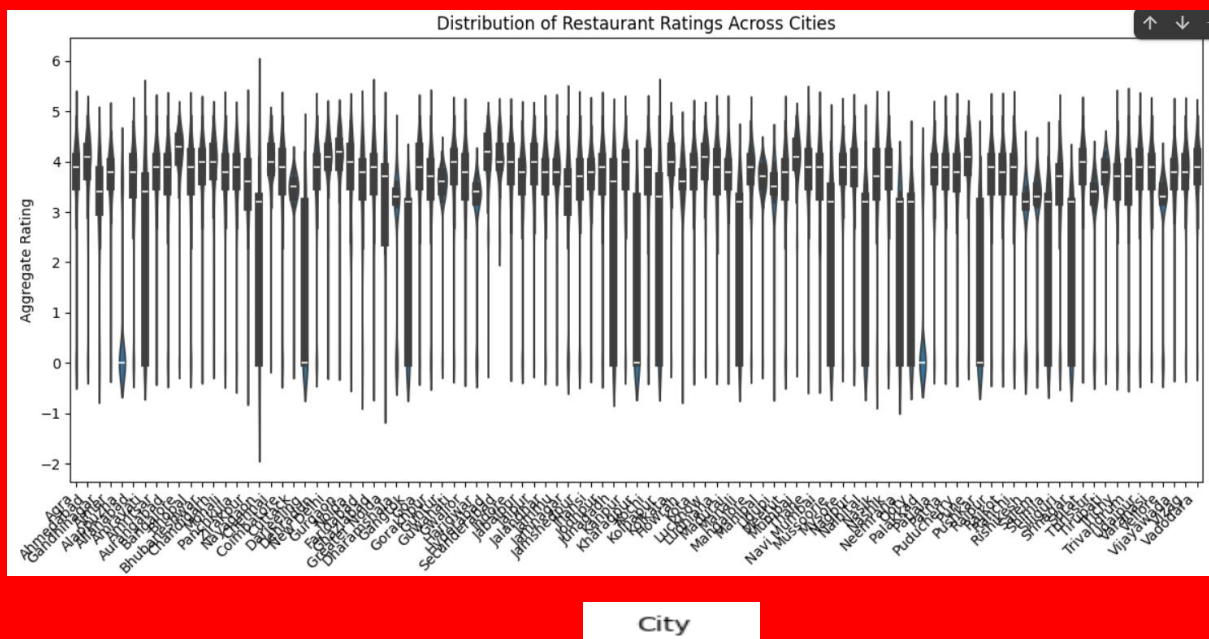
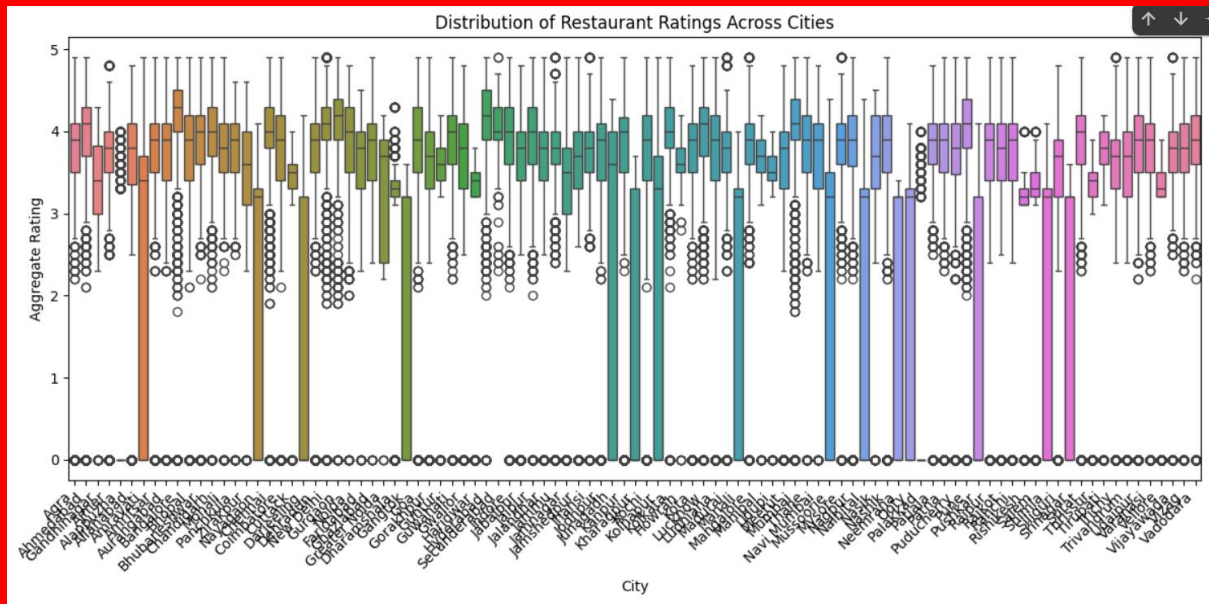
GROUP DATA BY CITY AND GET THE DISTRIBUTION OF AGGREGATE RATINGS.

RESET THE INDEX FOR BETTER PLOTTING.

CREATE A BOX PLOT TO VISUALIZE THE DISTRIBUTION

```
plt.figure(figsize=(12, 6))
sns.violinplot(x='city', y='aggregate_rating', data=df)
plt.title('Distribution of Restaurant Ratings Across Cities')
plt.xlabel('City')
plt.ylabel('Aggregate Rating')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

ALTERNATIVELY, CREATE A VIOLIN PLOT FOR A MORE DETAILED DISTRIBUTION VIEW.



CUISINE ANALYSIS:

DETERMINE THE MOST POPULAR CUISINES AMONG THE LISTED RESTAURANTS.

```
df['cuisines'] = df['cuisines'].str.split(',')
df = df.explode('cuisines')

df['cuisines'] = df['cuisines'].str.strip()

cuisine_counts = df['cuisines'].value_counts()

top_cuisines = cuisine_counts.head(10)

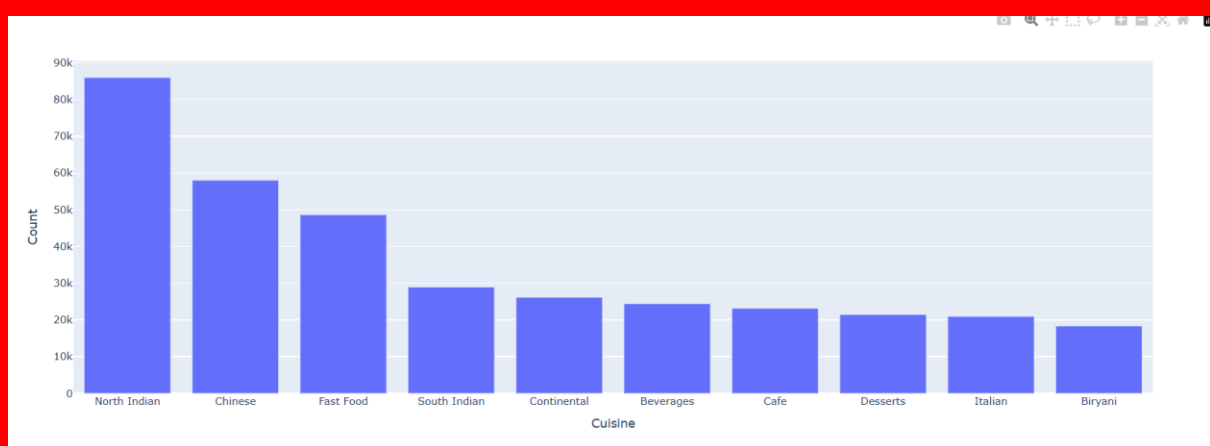
print(top_cuisines)

fig = px.bar(x=top_cuisines.index, y=top_cuisines.values, labels={'x': 'Cuisine',
                                                                'y': 'Count'})
fig.show()
```

cuisines	
North Indian	85965
Chinese	57989
Fast Food	48584
South Indian	28895
Continental	26126
Beverages	24382
Cafe	23140
Desserts	21437
Italian	20920
Biryani	18315

Name: count, dtype: int64

- 1.Explode the 'cuisines' column to individual cuisines.
2. Remove leading/trailing whitespace from cuisine names.
3. Count the occurrences of each cuisine.
4. Get the top N most popular cuisines (e.g., top 10)
5. Print or visualize the results.



We can also create a bar chart to visualize the top cuisines:

This code generates a bar chart using Plotly Express to visualize the top cuisines and their counts. The x-axis represents the different cuisines, and the

y-axis represents their respective counts. Custom labels are applied to the axes, with "Cuisine" for the x-axis and "Count" for the y-axis.

CUISINE ANALYSIS:

INVESTIGATE IF THERE'S A CORRELATION BETWEEN THE VARIETY OF CUISINES OFFERED AND RESTAURANT RATINGS.

```
df['cuisines'] = df['cuisines'].str.split(',')
df = df.explode('cuisines')

df['cuisines'] = df['cuisines'].str.strip()

cuisine_ratings = df.groupby('cuisines')['aggregate_rating'].mean().reset_index()
cuisine_ratings = cuisine_ratings.sort_values(['aggregate_rating', ascending=False])
print(cuisine_ratings)

fig = px.scatter(cuisine_ratings, x='cuisines', y='aggregate_rating',
                 labels={'cuisines': 'Cuisine', 'aggregate_rating': 'Average Rating'},
                 title='Correlation between Cuisine Variety and Restaurant Rating')
fig.show()

df['cuisine_count'] = df.groupby('name')['cuisines'].transform('nunique')

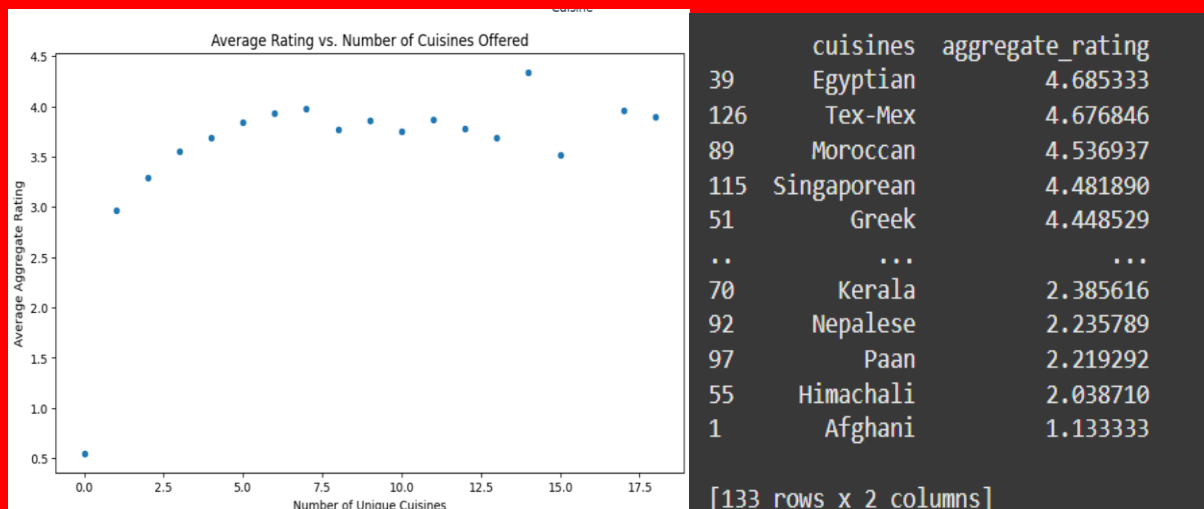
cuisine_count_ratings = df.groupby('cuisine_count')['aggregate_rating'].mean().reset_index()

plt.figure(figsize=(10, 6))
sns.scatterplot(x='cuisine_count', y='aggregate_rating', data=cuisine_count_ratings)
plt.title('Average Rating vs. Number of Cuisines Offered')
plt.xlabel('Number of Unique Cuisines')
plt.ylabel('Average Aggregate Rating')
plt.show()

correlation = cuisine_count_ratings['cuisine_count'].corr(cuisine_count_ratings['aggregate_rating'])
print(f"Correlation between number of cuisines and average rating: {correlation}")
```

1. Explode the 'cuisines' column to individual cuisines.
2. Remove leading/trailing whitespace from cuisine names.
3. Group by cuisines and calculate the mean aggregate rating for each cuisine.
4. Sort cuisines by average rating in descending order.
5. Print the top cuisines and their average ratings.
6. Visualize the correlation (e.g., using a scatter plot).

7. Calculate the number of unique cuisines offered by each restaurant.
8. Group by cuisine count and calculate the average rating.
9. Plot the relationship
10. Calculate the correlation.



PRICE RANGE AND RATING:

Analyze the relationship between price range and restaurant ratings.

```
plt.figure(figsize=(10, 6))
vb=px.scatter(df,x='price_range', y='aggregate_rating',color='price_range')

vb.show()

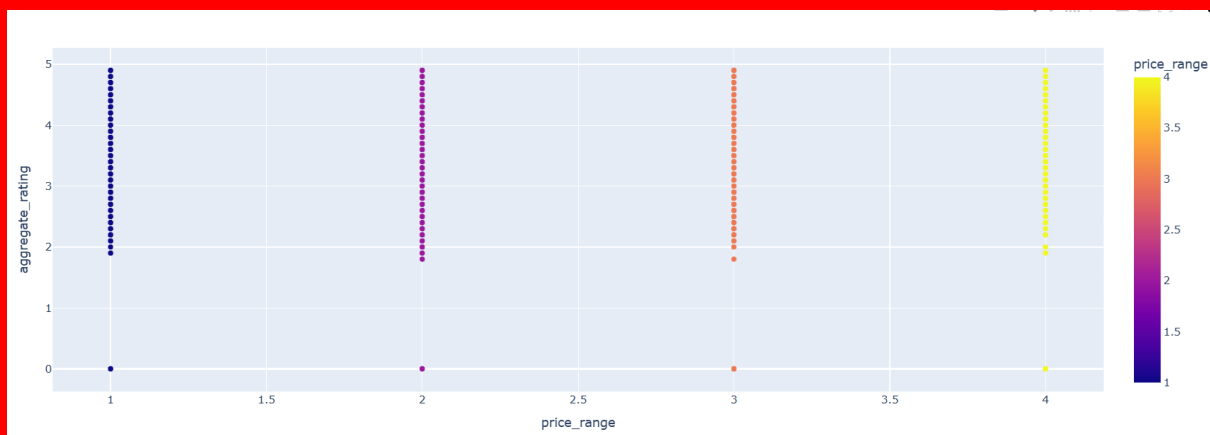
correlation = df['price_range'].corr(df['aggregate_rating'])
print(f"The correlation coefficient between price range and aggregate rating is: {correlation}")

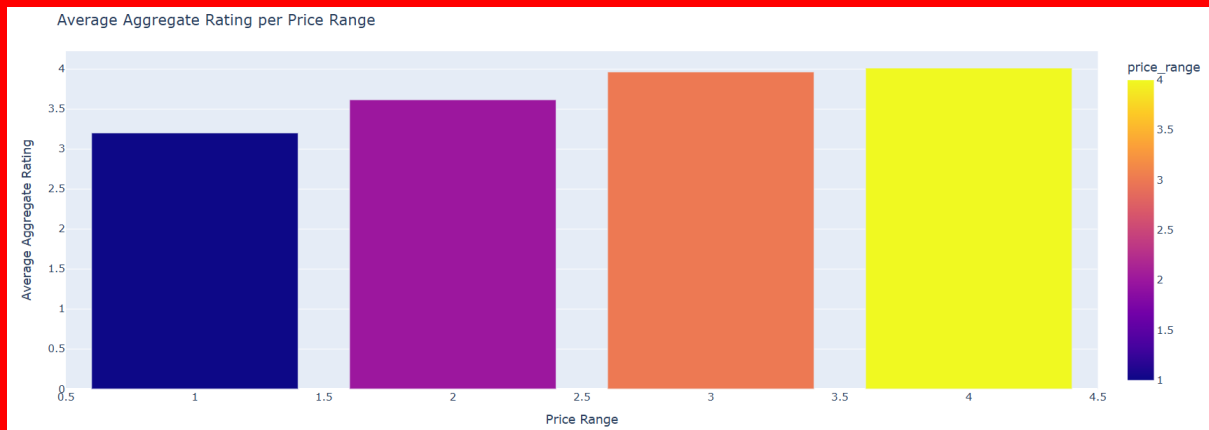
price_range_ratings = df.groupby('price_range')['aggregate_rating'].mean().reset_index()

plt.figure(figsize=(8, 6))
baar=px.bar(price_range_ratings,x='price_range', y='aggregate_rating',color='price_range')
baar.update_layout(
    title='Average Aggregate Rating per Price Range',
    xaxis_title='Price Range',
    yaxis_title='Average Aggregate Rating')

baar.show()
```

This code visualizes the relationship between `price_range` and `aggregate_rating` from a dataset. It first creates a scatter plot to display how these variables are related, followed by calculating and printing the correlation coefficient. Finally, a bar chart is generated to show the average `aggregate_rating` for each `price_range` category.





VISUALIZE THE AVERAGE COST FOR TWO PEOPLE IN DIFFERENT PRICE CATEGORIES.

```
average_cost_by_price_range = df.groupby('price_range')['average_cost_for_two'].mean().reset_index()
fig = px.bar(average_cost_by_price_range,
             x='price_range',
             y='average_cost_for_two',
             color='price_range',
             labels={'price_range': 'Price Category', 'average_cost_for_two': 'Average Cost for Two'},
             title='Average Cost for Two People Across Different Price Categories')
fig.show()
```

This code first groups the DataFrame `df` by the `price_range` column and computes the mean of `average_cost_for_two` for each price category. The result is stored in a new DataFrame, `average_cost_by_price_range`. Using Plotly's `px.bar()`, it creates a bar chart where the x-axis



represents different price categories and the y-axis shows the average cost for two people. The chart is customized with labels for the axes and a title for clarity. Finally, the `fig.show()` method displays the chart.

TOP RESTAURANT CHAINS:

IDENTIFY AND VISUALIZE THE TOP RESTAURANT CHAINS BASED ON THE NUMBER OF OUTLETS.

```
a=df.groupby('name')['aggregate_rating'].count().reset_index()
a.rename(columns={"aggregate_rating":"outlet_count"},inplace=True)
a=a.sort_values("outlet_count",ascending=True)
b=a.nlargest(10, 'outlet_count')
b.reset_index()
```

	index	name	outlet_count
0	10302	Domino's Pizza	406
1	6054	Cafe Coffee Day	323
2	17982	KFC	261
3	34434	Subway	211
4	18820	Keventers	208
5	3885	Baskin Robbins	207
6	22401	McDonald's	155
7	27096	Pizza Hut	150
8	5700	Burger King	144
9	3787	Barbeque Nation	120

```
fig=px.bar(b,x="name",y="outlet_count",color="name")
fig.show()
```



EXPLORE THE RATINGS OF THESE TOP CHAINS.

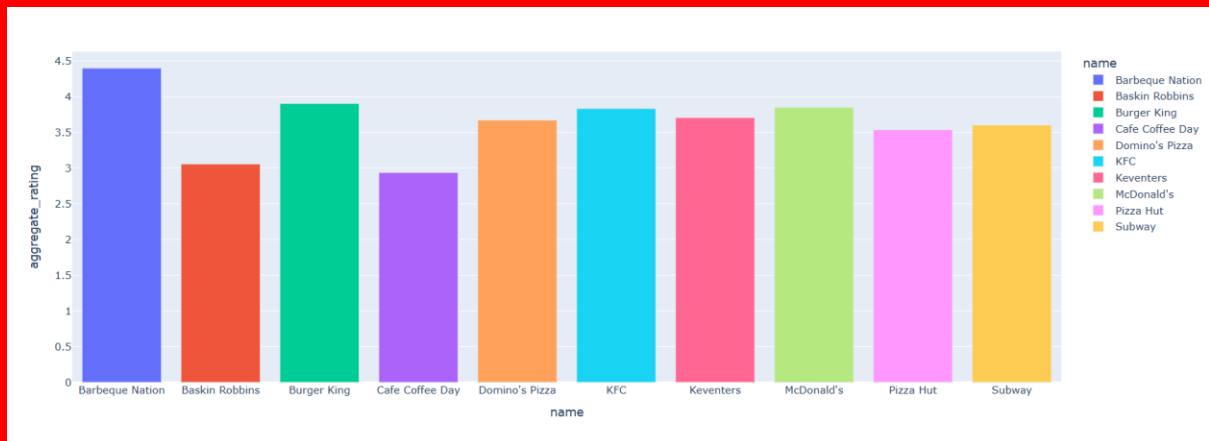
```
top_chains = b['name'].tolist()
top_chains
```

```
["Domino's Pizza",
 'Cafe Coffee Day',
 'KFC',
 'Subway',
 'Keventers',
 'Baskin Robbins',
 "McDonald's",
 'Pizza Hut',
 'Burger King',
 'Barbeque Nation']
```

```
top_five_rating=df[df['name'].isin(top_chains)].groupby("name")["aggregate_rating"].mean().reset_index()
top_five_rating
```

	name	aggregate_rating
0	Barbeque Nation	4.397500
1	Baskin Robbins	3.054106
2	Burger King	3.902083
3	Cafe Coffee Day	2.935294
4	Domino's Pizza	3.669212
5	KFC	3.831418
6	Keventers	3.704808
7	McDonald's	3.849032
8	Pizza Hut	3.535333
9	Subway	3.600474

```
top=px.bar(top_five_rating,x="name",y="aggregate_rating",color="name")
top.show()
```



WORD CLOUD FOR REVIEWS:

Create a word cloud based on customer reviews to identify common positive and negative sentiments. Analyse frequently mentioned words and sentiments

```
from wordcloud import WordCloud, STOPWORDS
```

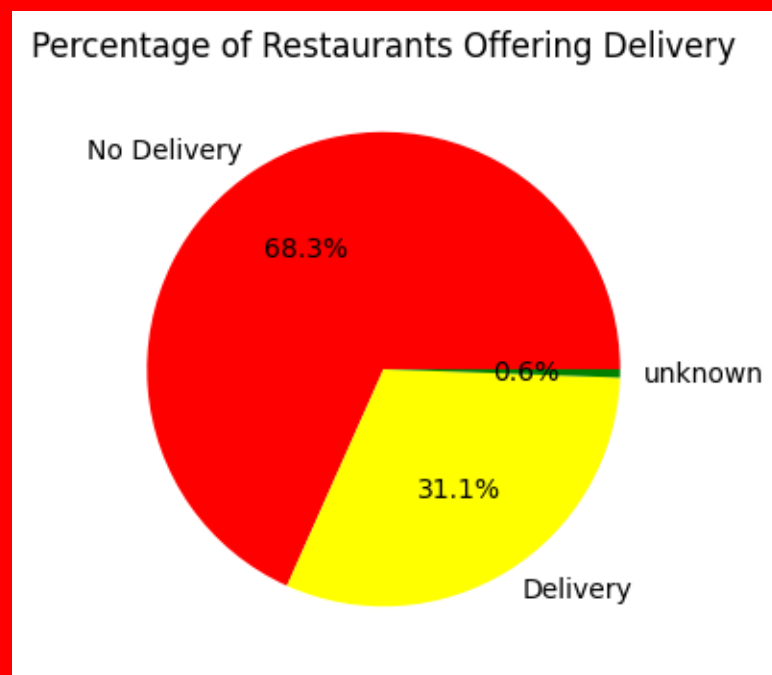
```
text=" ".join(review for review in df.rating_text)
wordcloud=WordCloud(width=800,height=400,background_color="white",stopwords=STOPWORDS).generate(text)
plt.figure(figsize=(8,8),facecolor=None)
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad=0)
plt.show()
```



ONLINE ORDER AND TABLE BOOKING:

INVESTIGATE THE IMPACT OF ONLINE ORDER AVAILABILITY ON RESTAURANT RATINGS. ANALYZE THE DISTRIBUTION OF RESTAURANTS THAT OFFER TABLE BOOKING.

```
plt.figure(figsize=(8, 4))
delivery_counts = df['delivery'].value_counts()
labels = delivery_counts.index.tolist()
label_mapping = {-1: 'No Delivery', 0: 'unknown', 1: 'Delivery'}
labels = [label_mapping.get(label, label) for label in labels]
plt.pie(delivery_counts, labels=labels, autopct='%1.1f%%', colors=['red', 'yellow', 'green'])
plt.title("Percentage of Restaurants Offering Delivery")
plt.show()
```



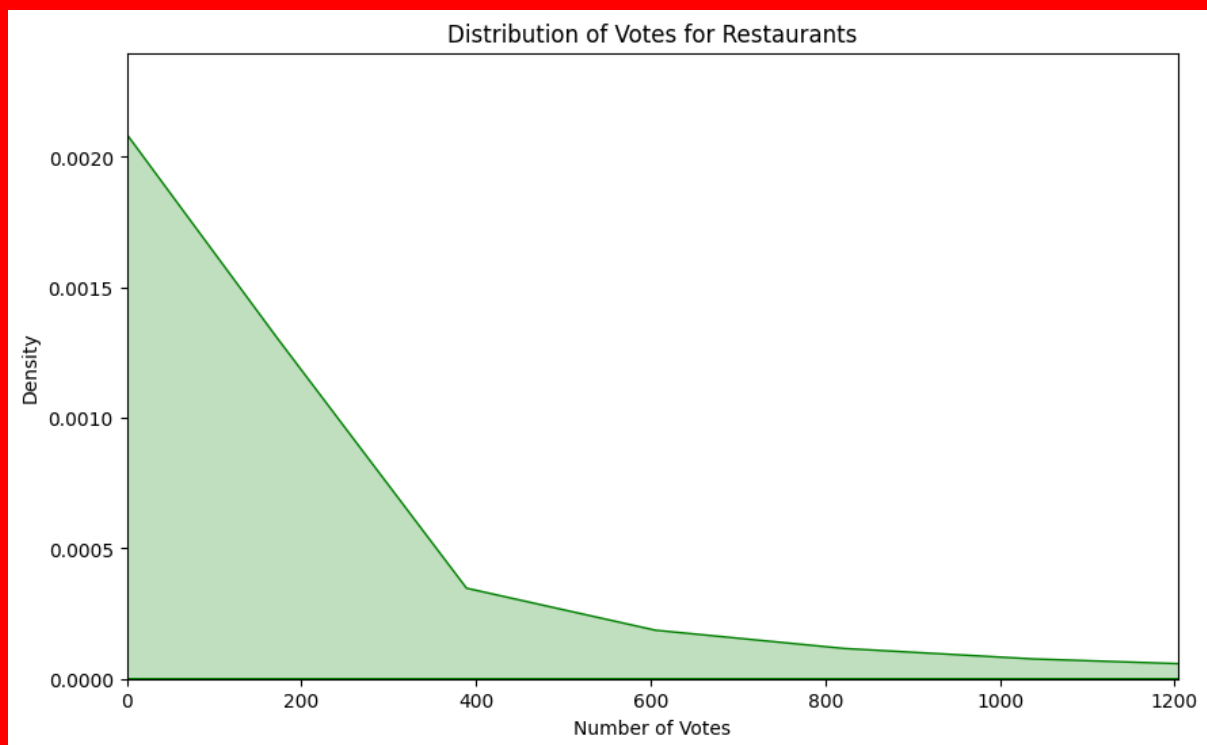
RESTAURANT FEATURES:

ANALYZE THE DISTRIBUTION OF RESTAURANTS BASED ON FEATURES LIKE Wi-Fi, ALCOHOL AVAILABILITY, ETC. INVESTIGATE IF THE PRESENCE OF CERTAIN FEATURES CORRELATES WITH HIGHER RATINGS.

```
plt.figure(figsize=(10, 6))
sns.kdeplot(df["votes"], shade=True, color="green")

plt.xlabel("Number of Votes")
plt.ylabel("Density")
plt.title("Distribution of Votes for Restaurants")
plt.xlim(0, df["votes"].quantile(0.95)) # Limit extreme values for better visualization
plt.show()
```

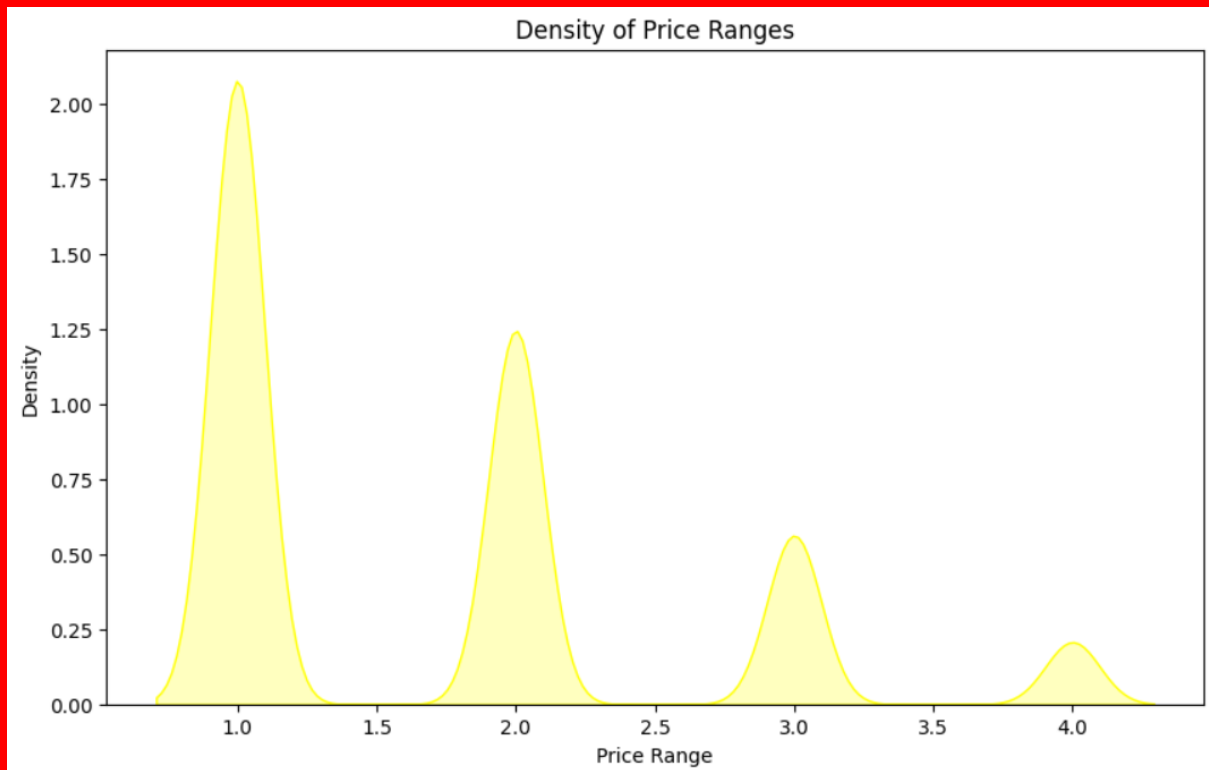
DISTRIBUTION OF VOTES FOR RESTAURANTS



DENSITY OF PRICE RANGE

```
plt.figure(figsize=(10, 6))
sns.kdeplot(df["price_range"], shade=True, color="yellow")

plt.xlabel("Price Range")
plt.ylabel("Density")
plt.title("Density of Price Ranges")
plt.show()
```

conclusion

❑Major Cities Dominate the Restaurant Industry 🍽️

.The highest number of restaurants are located in major metro cities like Delhi, Mumbai, Bangalore, and Hyderabad.

.This indicates that urban areas have a high demand for dining options, making them prime locations for new restaurants.



2 Indian Cuisine is the Most Popular, But Other Cuisines Are Growing 🍛🍽️

.The most frequently served cuisines include North Indian, Chinese, South Indian, and Fast Food.

.International cuisines like Italian, Continental, and Mexican are also gaining popularity, indicating a diverse food culture in India.

3 Higher Price Doesn't Always Mean Higher Ratings 💰★

.The analysis showed that restaurants in higher price ranges do not always have the highest ratings.

.Mid-range restaurants (Price Range 2 & 3) tend to receive better ratings compared to the most expensive ones.

.This suggests that good food and service matter more than high prices for customer satisfaction.

4 Most Restaurants Have Moderate to High Ratings 📊

.The majority of restaurants have ratings between 3.5 to 4.5, showing that customers generally have a good dining experience.

.Very few restaurants received low ratings (<2.5), indicating that low-quality places struggle to survive in the competitive market.

5 Online Food Delivery is a Growing Trend 🚚

.A large percentage of restaurants now offer delivery services, reflecting the rise in online food ordering via Zomato, Swiggy, etc.

.This trend suggests that cloud kitchens and delivery-focused restaurants are becoming more profitable.

6 Bangalore Has the Most 5-Star Restaurants 🌟

.When analyzing top-rated restaurants (5-star ratings), Bangalore had the most highly-rated places.

.This shows a strong preference for high-quality dining in tech hubs and metro cities.



If we go by numbers, Zomato witnessed a record rise of 225% in its revenue in the very first half of FY2020. The company's biannual report revealed that it had registered \$205 Mn in revenue as compared to \$63 Mn (in the first half) of last year. Zomato's journey thus far has witnessed crests and troughs of its own. But it wouldn't be wrong to say that ups and downs are faced by almost all startup companies in their initial years. What really matters, in the end, is their resilience, their will to stand by the organization and its values and the decisions they make in difficult situations. As for the founders, Deepindar and Pankaj, they've done a phenomenal job of keeping Zomato afloat even in times of crisis. Where in early 2018, Pankaj announced that he'd like to quit Zomato and take some time off before he turned "professionally active" again, he continues to hold a stake of 3.11% in the organization. Today, the company is looking forward to raising more funds with a vision to expand its service portfolio. It aims to reach newer markets and explore other related domains of growth. As Zomato battles it out with Swiggy - its biggest competitor, it will be interesting to see what lies ahead for this organization especially in a time when bigger players are entering the market.



Zomato

24 July 2019 · 🌐

...

Get up to 50% OFF on your first 5 orders on Zomato! Also applicable on Cash on Delivery.



Savory Special Thali

Install Now



Crunchy Masala Dosa

👍 74K

1.9K comments · 417 shares

👍 Like

💬 Comment

➦ Share

